

Good evening everyone



class start at 9:08

HashMap → synchronised → concurrent.

map
string → int

multi thread environment

th1 → get("a")

th2 → remove("c")

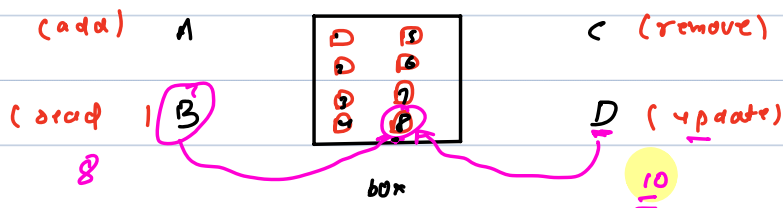
th3 → add("z", 105)

th4 → update("a",
50);

"a"	-	101
b	-	305
c	-	298
d	-	36
e	-	49
f	-	174
g	-	234
h	-	937
i	-	238

thread is not
safe.

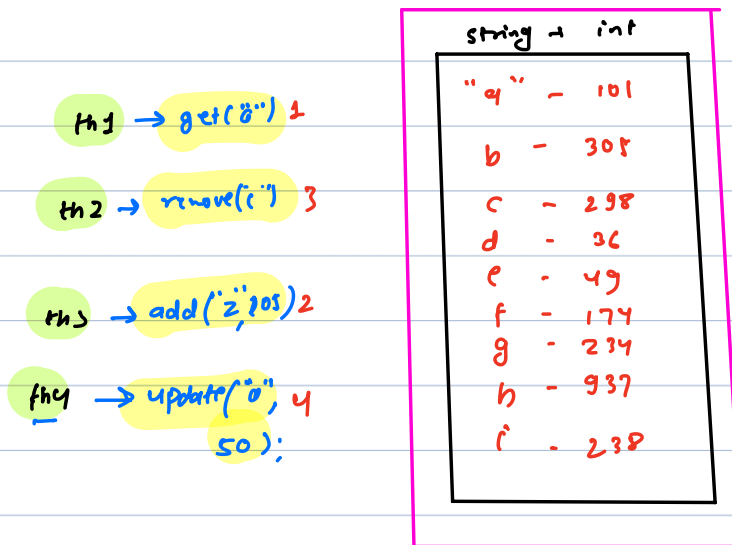
all threads are trying to enter in the map at
same time.



chances of race conditions.

inconsistency occur when multiple threads enter
at same time.

we make this synchronized so thread can be safe.

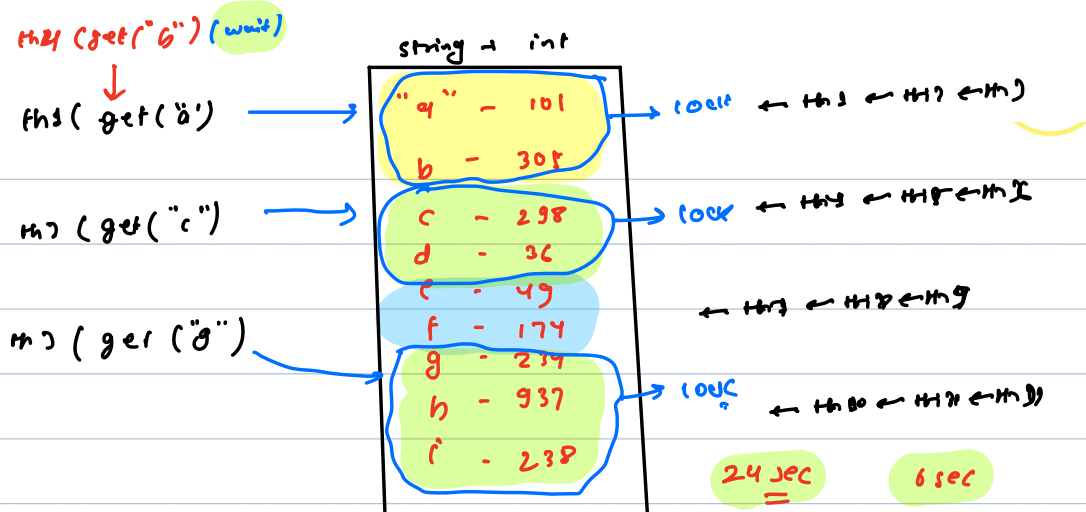


- only one thread can modified the map at a time
- In this we lock the entire map when a thread is enter.
- it is not support multi threading.

if it a thread takes 2 sec and we have 20 threads.

40 sec

concurrent hashmap



it is support multithreading environment.

multiple thread can execute simultaneously.

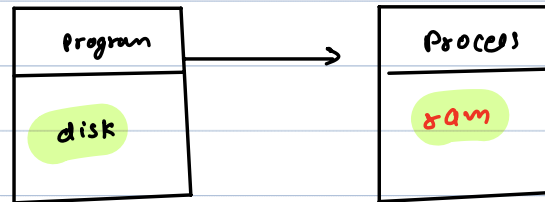
its improve overall system.



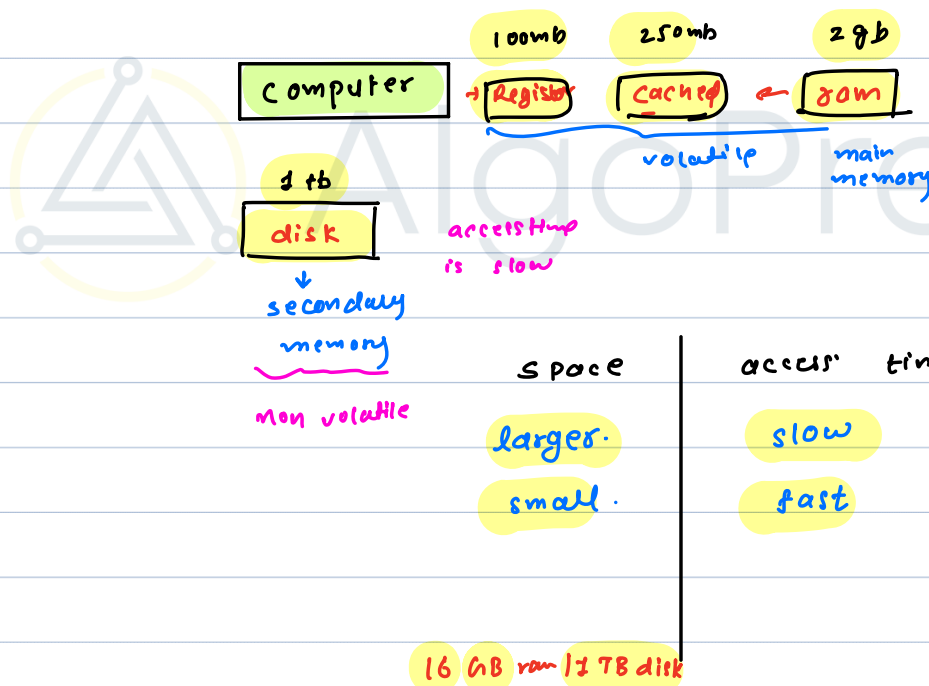
AlgoPrep

memory management

how data is stored in process



cpu execute the process.



• disk contains huge space and access time is slow.

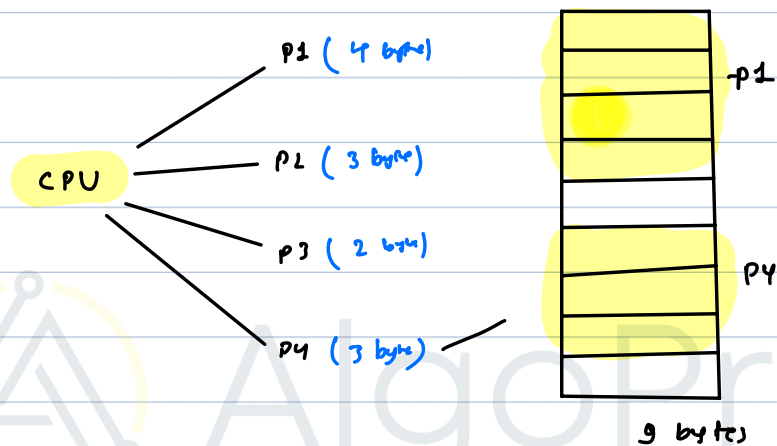
• CPU can't talk directly to the disk

• access time is fast in register due to small space.

• cache is faster than ram and slower than register

- ram are volatile in nature. if we shut down the computer then data is erased.
- disk is non volatile.
- data stored in disk is persist even if we power off the computer

How data is stored in memory.

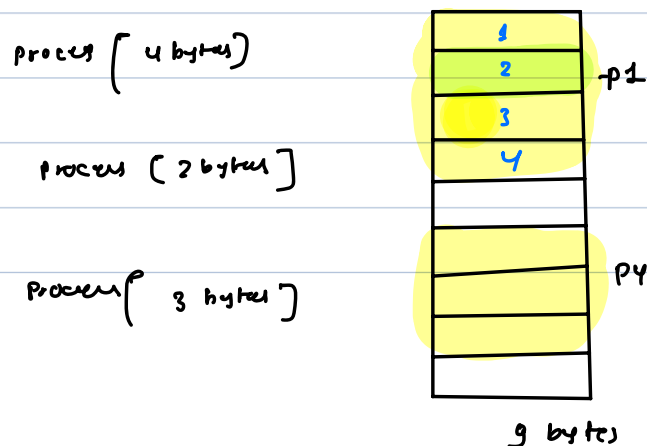


CPU load the process into ram to execute.

How we manage the memory allocation.

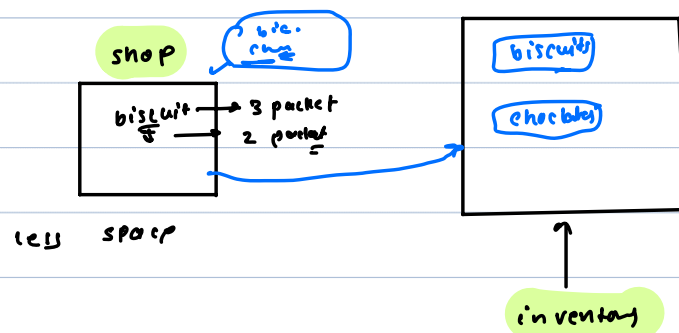
1) Contiguous memory allocation:-

each process is stored in a single contiguous block of memory to store in entire address space.

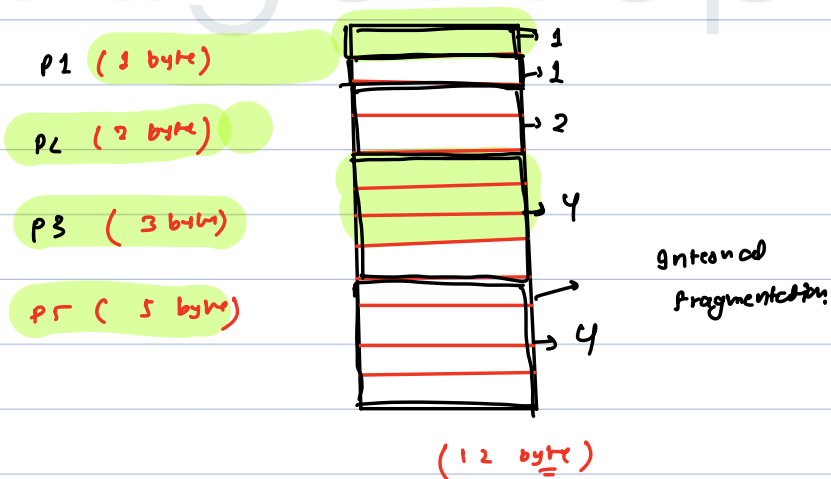


- we can't break any process.
- it is easy to implement.

- when we ask to CPU to get some data then it takes nearby value.

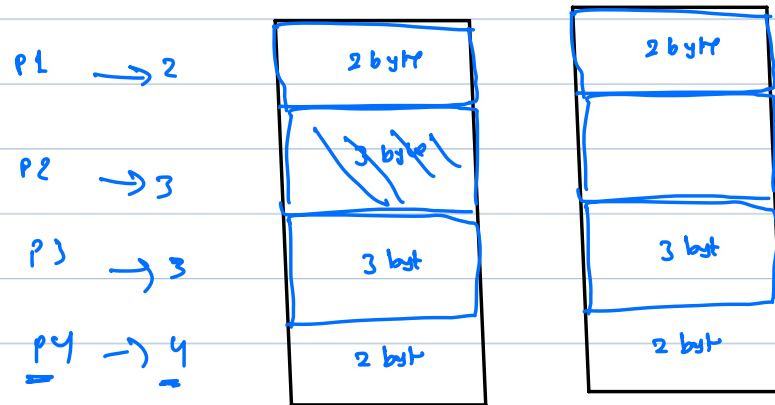


Fixed memory allocation:

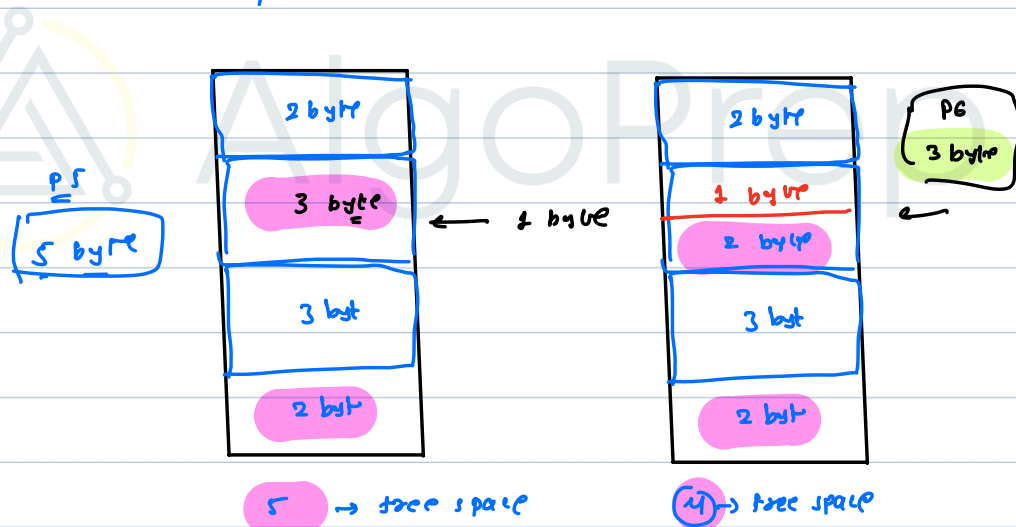


- we predetermine the size of each part and cannot change further.
- wastage of storage
- internal fragmentation occurs, where free memory is divided into small parts, unusable blocks between allocated blocks

variable memory allocation.



when a process is ended it creat a hole.



- operating system manages memory dynamically.
- dividing this available memory into variable size portion. based on process requirements.
- external fragmentation occur and also wastage memory.

it is better than the fixed memory allocation.

break till 10:20

if a game needs 8gb ram and we have only 6 gb ram.

IDE → consume lot space

google chrome →

How we will do this?

Paging :- it is a technique of memory management to efficiently manage memory. by dividing it into fixed size of blocks called pages.

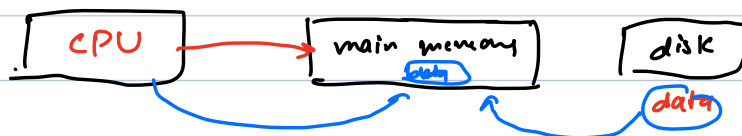
process

8 gb

6 gb

• whatever space is available in your ram we this and rest of the part is - stored in disk.

• if CPU is tried to access some data which is present in the disk then first it brought to the main memory then execute by CPU



everything in this is handle by MMU

memory management unit

logical
address

{ virtual address
of data }

physical
address

{ exact location
of data }

- when a program is running a logical address is generated and we use this to reach the physical address.

we create a table to map the logical address and physical address and this table is called Page table.

PT [16 bytes]

pin	bytes	4 byte →		
0	00	0-4	4	00
1	03	5-8	4	01
2	04	9-12	4	02
3	06	13-16	4	03

00	P0 → 0	4
01		4
02		4
03	P0 → 1	4
04	P1 → 2	4
05		4
06	P1 → 3	4
07		4
08		4

size of pages is
equal to size of frame.

Ram frame

it store in non continuous form.

when we divide memory, part is called frames



we make different page table for each process.

give me data of 7 byte.

to get the page number → $7/4 = 1$

offset → $7 \% 4 = 3$ → byte number

if the data is stored in disk then mmy
brough the data from the disk and allocate
in main memory if space is available.

Replacement algorithm